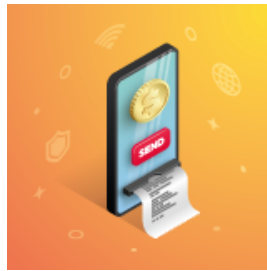


IAPGUARD Unity SDK



A sample project for the Unity game engine, that contains fully working in-app purchase implementations including server-side receipt validation for the Apple App Store and Google Play. The [IAPGUARD Service](#) is a fully hosted solution, so you do not have to manage your own server or keep track of transactions and App Store API changes. The platform offers a FREE plan to get started.

Requirements:

- You have created a free account on the [IAPGUARD Platform](#)
- You have created a new app in the IAPGUARD Dashboard
- You should have set up your app and products on the App Store(s) already prior to running this project

Table of Contents

1. [Project Setup](#)
2. [Installation](#)
3. [Usage](#)
4. [Testing](#)
5. [Integration](#)
6. [Missing Features](#)
7. [Support](#)

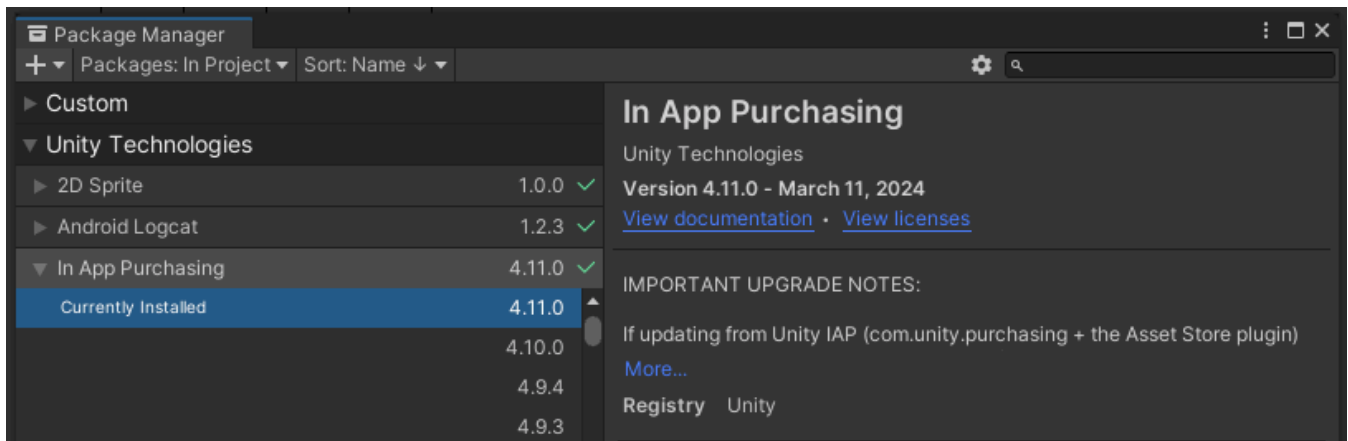
Project Setup

Google Play: You should have created a [Google Play Developer Account](#), added an application and created in-app purchases for it.

Apple App Store: You should have enrolled in the [Apple Developer Program](#), added an application and created in-app purchases for it.

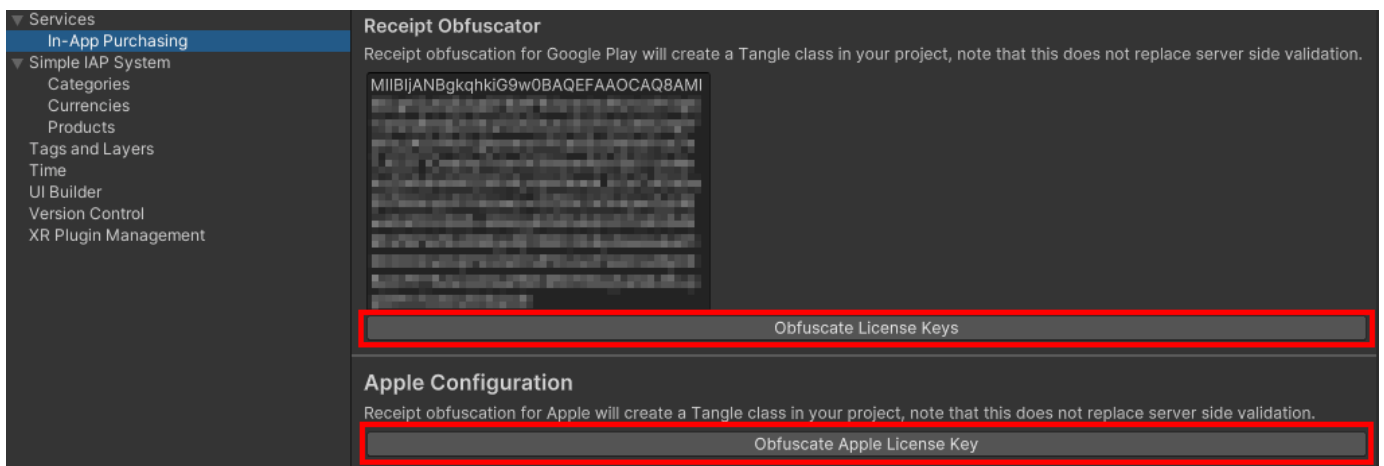
- Verify that your project's Package Name matches your App Store's bundle identifier in `Edit > Project Settings > Player > Other Settings > Identification > Package Name` .

This project requires [Unity IAP](#) for billing. Import the latest Unity IAP package in your project via `Window > Package Manager` .



Because this project uses both local and server-side validation, follow the next steps for creating your secret files for local validation in Unity IAP.

- Open Edit > Project Settings > Services > In-App Purchasing
- Make sure that the toggle in the top-right corner is enabled (or enable it)
- Scroll down to the Receipt Obfuscator section
 - Google Play: find your Google Play Public Key in your Google Play Developer Console under Your App > Monetise > Monetisation setup > Google Play Billing > Base64-encoded RSA public key
 - Apple App Store: no setup necessary
- Click the Obfuscate buttons to generate your Google Play and Apple Tangle files



NOTE: You can now scroll back up and turn off the In App Purchases service again! This does not affect purchasing in any way, it just prevents sending analytics to Unity in case you do not intend to use that.

Installation

Get this plugin from the [Unity Asset Store](#) or from GitHub and import it into your project.

Usage

The `Demo` scene provides buttons for buying each type of in-app product, along with a way to restore transactions. The logic for this is in the `UIDemo` script.

CONSUMABLE
Example: boost or powerups



NON CONSUMABLE
Example: new car or race track



SUBSCRIPTION
Example: daily bonus rewards



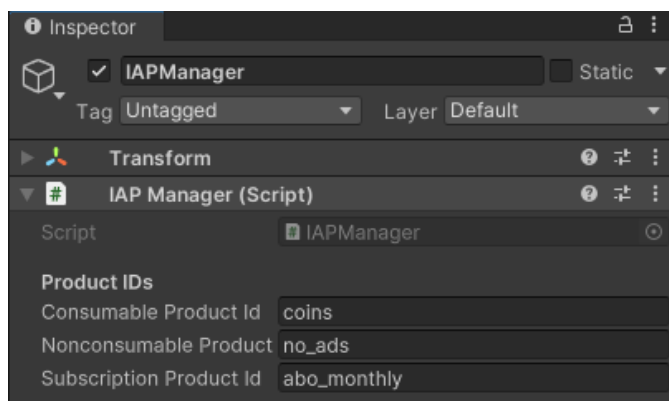
RESTORE
Resend existing receipts



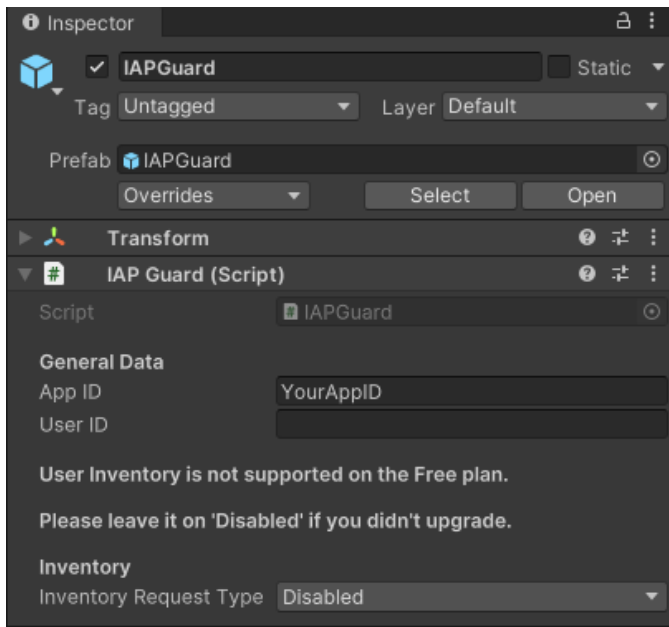
SHOW LOG

Please test on a mobile device. Editor purchases do not generate any receipts.

If you wish to use your own products in the Demo scene, select the `IAPManager` game object in the Hierarchy and modify the existing ones. The `ID` you enter here is the product identifier from the App Store.



Likewise, select the `IAPGuard` game object and enter your own **App ID** from the IAPGUARD platform.



Testing

Since the Unity Editor does not have any connection to App Stores, you are not able to test purchases using it. In the `Demo` scene, a message is shown mentioning this too. You have to test on a real device that is actually able to receive valid receipts and transactions from the App Stores.

PLEASE NOTE: Guides on how to upload your application to the App Stores, or creating test users for Sandbox purchasing is out of scope for this document. If you need help with that, please contact us (see [Support](#)).

When running on a real device you will be able to fully test all purchase validation workflows. Press the **SHOW LOG** button to display an informative console about web requests. In the following screenshot, a consumable and non-consumable product have been bought and validated. If you make use of IAPGUARD's User Inventory feature, the non-consumable product will be returned for that user in subsequent app launches as well.



Integration

New Project

IAPManager: You can either rewrite the `IAPManager` script to fit into your game coding, or use it as-is. This script offers a **`purchaseCallback`** event that you would want to subscribe to, in order to define what happens when a purchase event is received. Check that the *success* boolean is true before accessing the *data* `JSONNode` and granting the corresponding product to the player. Note that the *data* `JSONNode` will return only the product ID, if the purchase was restored, or if server validation was disabled or does not support the build platform.

It is recommended to save successful purchases in an encrypted file locally, so you can access them at any time and players can also use them when playing offline.

IAPGuard: The `IAPGuard` script offers a **`purchaseCallback`** event for the web response as well - when using your own `IAPManager`, it should subscribe to this. Also, an **`inventoryCallback`** event that returns active purchases. If you are making use of `IAPGUARD`'s User Inventory feature, you would subscribe to this event after player authentication and `IAPGuard` initialization, call `IAPGuard.RequestInventory()` and act on the products and status it returns. In this demo, if enabled (requires paid plan), the `UIDemo` script checks User Inventory and sets the purchase status of the products accordingly.

Existing Project

For an integration in an existing project that already implements Unity IAP, the following points need to be done:

1. Instantiate the `IAPGuard` prefab
2. Initialize the `IAPGuard` component (`Initialize`)
3. Extend the `ProcessPurchase` method with validation (`RequestPurchase`)
4. Link `IAPGuard`'s `purchaseCallback` Action

5. (optional) Add User Inventory (`RequestInventory` , `GetInventory`)
6. Add a way to let users restore their transactions (`RequestRestore`)

1. Place the Prefabs > IAPGuard prefab into the first scene of your application. It calls `DontDestroyOnLoad` on itself and therefore persists across scene changes, but is not initialized until you do so.

2. In your Unity IAP handler's code:

- add the asset's namespace at the top:

```
using FLOBUK.IAPGUARD;
```

- when you call `ConfigurationBuilder.Instance` to create a store module, save a reference to the `ConfigurationBuilder` in a variable as we will need it in the next step. In the `IAPManagerDemo` script, this is done in `Initialize()`.
- in your `OnInitialized` implementation, initialize the `IAPGuard` component by passing in the `IStoreController` and `ConfigurationBuilder` reference stored previously:

```
IAPGuard.Instance.Initialize(controller, builder);
```

3. In your `ProcessPurchase` implementation, pass the received product for validation to `IAPGUARD` and get its `PurchaseState`:

```
PurchaseState state = IAPGuard.Instance.RequestPurchase(product);
```

If the `PurchaseState` is `Pending` , it is important that you return `PurchaseProcessingResult.Pending` to keep the transaction open. If the transaction was processed, we will receive a callback from `IAPGUARD` in the next step. Otherwise, the transaction was completed. You will then want to reward the user in case the `PurchaseState` is `Purchased` (not in case it is `Failed`) and return `PurchaseProcessingResult.Complete` .

4. In the previous step, we already rewarded the user if the transaction was complete. However, we only handled the Unity IAP part, in case `IAPGUARD` was not used. Now, we have to implement the `IAPGUARD` callback when finishing a transaction too. In `OnInitialized` , below the initialization call add:

```
IAPGuard.purchaseCallback += OnPurchaseResult;
```

To keep things simple, in the `IAPManager` demo script we defined a separate `OnPurchaseResult` method that is also used in the `ProcessPurchase` implementation. This means that both interfaces call the same method to reward the user.

5. **PAID PLAN ONLY:** if you make use of user authentication and inventory, a user's purchases are stored in the backend. To retrieve them, on the `IAPGuard` prefab set the `Inventory Request Type` to a value other than `Disabled` and add the following line at the end of `OnInitialized` :

```
IAPGuard.Instance.RequestInventory();
```

You can either retrieve the inventory by subscribing to the `IAPGuard.inventoryCallback` action or by calling `IAPGuard.Instance.GetInventory()` manually later.

6. You will want to allow users to manually restore their purchases and re-sync them with the `IAPGUARD` backend, in case they switch devices, user IDs or lost their local storage in other ways. Add the below method

and let the user call it manually.

```
public void RestoreTransactions()
{
    #if UNITY_IOS
        extensions.GetExtension<IAppleExtensions>().RestoreTransactions(null);
    #else
        IAPGuard.Instance.RequestRestore();
    #endif
}
```

Missing Features

The purpose of this SDK is to quickly get up and running with in-app purchases and server-side receipt validation, while keeping it as simple as possible for you to extend and build upon.

A paid plugin for in-app purchases supporting:

- category, product and virtual currency management right in the Unity Editor
- automatic handling of virtual product purchases (for virtual currency)
- product overrides for different App Store IDs
- local storage of purchases for offline use
- several shop scenes for vertical or horizontal item alignment

in addition to built-in support for server-side receipt validation, is available on the [Unity Asset Store](#).

Support

This asset was specifically designed to be used in combination with [IAPGUARD](#). For integration assistance or questions about the Service, please open a **Support ticket** within the IAPGUARD Dashboard.